Eyad Mohammed Elghonemy
ID: 132300082

Task 1 — WatchIT Streaming Platform

1 Entity & Attribute Table

Fill in the table below with all 5 tables (4 entities + 1 junction), their attributes, and mark PKs and FKs:

Table Name	Attributes (mark PK & FK)	Notes
SubscriptionPlan	PlanID (PK), PlanName (NOT NULL), C	No FKs, first to create
Creator	CreatorID (PK), Name, Bio, JoinDate	
[User]	UserID (PK), Username, Email, PlanID	FK to row 2
Video	VideoID (PK), Title, Duration, CreatorI	Reserved word — bracket it!
Wathed	UserID (PK, FK), VideoID (PK, FK), Wat	Junction table — composite PK

2 Relationships & Cardinality Table

From	Verb	To	Cardinality	FK Location / Note
[User]	subscribes to	SubscriptionPlan	M:1	PlanID on [User].
Creator	produces	Video	1:M	CreatorID On Video.
[User]	watches	Video	M:M	Watched

3 CREATE TABLE Statements — WatchIT

Write all 5 CREATE TABLE statements in the correct order. Include all constraints.

```
    FullName    VARCHAR(100)    NOT NULL,
    Email       VARCHAR(100)    UNIQUE,
    JoinDate DATE,
    PlanID      INT     REFERENCES  SubscriptionPlan(PlanID)
);


CREATE TABLE Video(

    VideoID      INT PRIMARY KEY,
    VideoTitle   VARCHAR(100)    NOT NULL,
    Duration     INT,
    Genere       VARCHAR(100)    NOT NULL,
    ReleaseYear  INT,
    CreatorID    INT    REFERENCES  Creator(CreatorID)
);


CREATE TABLE Wathed(

    UserID      INT REFERENCES [User](UserID),
    VideoID     INT REFERENCES  Video(VideoID),
    Progress    INT,
    WatchedAt       DATETIME,
    PRIMARY KEY(UserID, VideoID)
);
```

⚠️ Remember: [User] must be bracketed. Watched needs a table-level PRIMARY KEY (UserID, VideoID).

4 ALTER TABLE — Add MaxDevices

Write the two ALTER TABLE statements to add MaxDevices (nullable first, then NOT NULL):

```
ALTER TABLE SubscriptionPlan ADD MaxDevices INT;

ALTER TABLE SubscriptionPlan ALTER COLUMN MaxDevices INT NOT NULL;
```

5 INSERT Rows — SubscriptionPlan & Watched

Write INSERT statements for SubscriptionPlan (at least 2 rows) and Watched (at least 3 rows):

```
  (2, 'Database Design 101', 60, 'Education', 2023, 1),
  (3, 'Cairo After Dark', 110, 'Drama', 2022, 2),
  (4, 'The Desert Code', 95, 'Thriller', 2024, 2);

INSERT INTO [User] (UserID, FullName, Email, Phone, JoinDate, PlanID)
VALUES
  (1, 'Ahmed Hassan', 'ahmed@watchit.io', '01012345678', '2024-01-10', 2),
  (2, 'Sara Mohamed', 'sara@watchit.io', '01098765432', '2024-02-20', 3),
  (3, 'Omar Khaled', 'omar@watchit.io', NULL, '2024-03-05', 1);


INSERT INTO Wathed(UserID, VideoID, WatchedAt, Progress)
VALUES
  (1, 1, '2024-04-01 20:00:00', 300),
  (1, 3, '2024-04-02 21:30:00', 620),
  (2, 2, '2024-04-03 19:00:00', 3200),
  (2, 4, '2024-04-04 22:00:00', 145),
  (3, 1, '2024-04-05 18:00:00', 1200);
```

Task 2 — Construction Company

1 Identify Entities & Attributes

List the 5 entity/table names and their attributes (mark PK, FK):

Table Name	PK Column	FK Columns (→ references)	Other Attributes
EMPLOYEE	EmpID	DeptCode (→ DEPT), Manag	EmpFName, EmpLName,
DEPARTMENT	DeptCode	EmpID (→ EMP)	DeptName, Address, Man
PROJECT	ProID	DeptCode (→ DEPT)	ProName, Location
WORKS_ON	EmpID + ProID	EmpID (→ EMP), ProID (→ P	WorkedHours
DEPENDENT	EmpID + FMName	EmpID (→ EMP)	Gender, Relationship

2 Explain the Circular FK Challenge

In your own words, explain why EMPLOYEE and DEPARTMENT create a circular FK problem and how you solve it:

```
ALTER TABLE Department ADD EmpID INT;
ALTER TABLE Department ADD CONSTRAINT fk_Emp FOREIGN KEY (EmpID) REFERENCES
Employee(EmpID);
```

3 CREATE TABLE — Construction Company (all 5 tables)

```
CREATE TABLE Project(

    ProID       INT     PRIMARY KEY,
    ProName     VARCHAR(100)    NOT NULL,
    Location    VARCHAR(100)    NOT NULL,
    DeptCode    INT     REFERENCES  Department(DeptCode)
);


CREATE TABLE FamilyMember(

    EmpID   INT REFERENCES Employee(EmpID),
    FMName  VARCHAR(100)    NOT NULL,
    Gender  VARCHAR(100)    NOT NULL,
    Relationdhip    VARCHAR(100)    NOT NULL,

    PRIMARY KEY(EmpID, FMName)
);

CREATE TABLE WorkedHours(

    EmpID   INT REFERENCES Employee(EmpID),
    ProID   INT REFERENCES Project(ProID),
    WorkedHours INT,
    PRIMARY KEY(EmpID, ProID)

);

CREATE TABLE EmpPhone(
    EmpID   INT REFERENCES Employee(EmpID),
    EmpPhone    INT,
    PRIMARY KEY(EmpID,EmpPhone)
);
```

⭐ The circular FK solution is the star of this task — CREATE EMPLOYEE (no DeptNum FK) → DEPARTMENT → ALTER EMPLOYEE ADD CONSTRAINT FK

4 Referential Integrity Constraints Table

Fill in all 7 FK relationships in the Construction Company schema:

Table	FK Column	References (Table.PK)	Meaning (1 sentence)
DEPARTMENT	MgrID	EMPLOYEE(EmpID)	The manager must be an
EMPLOYEE	DeptNum	DEPARTMENT(DeptCode)	Every employee must be a
EMPLOYEE	SupervisorID	EMPLOYEE(EmpID)	Self-referential — a super
PROJECT	DeptNum	DEPARTMENT(DeptCode)	Each project must be cont
WORKS_ON	EmpID	EMPLOYEE(EmpID)	Hours cannot be logged f
WORKS_ON	ProID	PROJECT(ProID)	Hours must be assigned t
DEPARTMENT	EmpID	EMPLOYEE(EmpID)	A dependent must belong

5 Bonus — INSERT 2 EMPLOYEE rows

Insert 2 employees. The first employee will have no supervisor (NULL) and no department yet. The second's supervisor is the first:

```
INSERT INTO EMPLOYEE (EmpID, EmpFName, EmpLName, Salary, ManagerID, DeptCode)
VALUES (1, 'Khaled', 'Nasser', 5000.00, NULL, NULL);

INSERT INTO EMPLOYEE (EmpID, EmpFName, EmpLName, Salary, ManagerID, DeptCode)
VALUES (2, 'Amira', 'Said', 4500.00, 1, NULL);
```

Note: DeptNum is NULL because DEPARTMENT doesn't exist yet (we haven't inserted departments). ALTER TABLE + INSERT departments first in real usage.